

Image Processing Project: Parallelization Performance Analysis

BUDURA Bogdan, DESHORS Victor

1 Repository Overview

This repository contains an image processing project implemented in C, featuring both sequential and parallelized versions. The project aims to compare performance differences between various parallelization techniques. Multiple implementation branches exist:

- `Using_Only_GPU`
- `Using_Only_MPI`
- `Using_Only_OpenMP`
- `Using_OpenMP_MPI_CUDA`

Our approach involved independent development of each parallelization method to evaluate which techniques are most effective for specific image processing operations. We then created an optimized mixed implementation combining the most effective strategies in the `Using_OpenMP_MPI_CUDA` branch.

Rather than parallelizing the entire pipeline at once, we chose to independently parallelize the filters, pixel loading, and storage functions. This methodical approach helped identify the most effective parallelization targets. An alternative strategy would be simultaneous parallelization of all three filters, potentially reducing overhead (especially for CUDA implementations).

1.1 Repository Structure

- **Image Folders:** Contains baseline, original, and processed images for both sequential and parallel methods
- **Source Code:** Includes sequential C files in `src/` and parallelized implementations in `src_para/`
- **Compilation Artifacts:** Object files stored in `obj/` (sequential) and `obj_para/` (parallel)
- **Key Files:**

- `Compare_runtime.py`: Performance comparison scripts
- Makefiles for compilation
- CSV logs tracking execution times
- **Execution Scripts:**
 - Sequential: `./run_test.sh`
 - Parallel: `./run_para.sh <nb_threads>`
- **Performance Tracking:** Runtime data logged in `durations_seq.csv` and `durations_para.csv` for detailed analysis

2 OpenMP Parallelization Strategy

Our OpenMP implementation targeted the following functions:

- `load_pixels`
- `apply_gray_filter`
- `apply_blur_filter`
- `apply_sobel_filter`

2.1 Analysis of `store_pixels` Function

Our attempted parallelization of the `store_pixels` function revealed several challenges that made efficient parallelization impractical:

1. Sequential color map construction with strong inter-iteration dependencies
2. Shared data structures requiring extensive synchronization
3. Complex nested loops with conditional logic
4. Critical sections for color identification creating parallel bottlenecks
5. Irregular memory access patterns poorly suited for data parallelism
6. Risk of non-deterministic output affecting image quality
7. Limited optimization potential given the small color space (maximum 256 colors)

While certain isolated sections could potentially benefit from parallelization, the algorithm’s inherently sequential nature would require substantial restructuring to achieve meaningful performance improvements. Based on this analysis, we maintained a sequential implementation for this function.

2.2 Implementation for load_pixels

We adopted a hierarchical parallelization approach for the `load_pixels` function:

2.2.1 Image-Level Parallelization

For GIFs containing multiple images, we distribute entire images across available threads using:

```
1 #pragma omp parallel for schedule(dynamic)
```

The dynamic scheduling handles potential load imbalance between images of varying complexity.

2.2.2 Pixel-Level Parallelization

We implemented conditional nested parallelism based on image size:

- For multi-image GIFs, large images (>100,000 pixels) receive additional pixel-level parallelization using nested OpenMP directives
- For single-image GIFs, pixel-level parallelization is conditionally applied when the image exceeds 50,000 pixels:

```
1 #pragma omp parallel for if(pixel_count > 50000)
2
```

2.2.3 Sequential Sections

Memory allocation and error handling remain sequential to ensure proper resource management, while computational pixel-processing operations are parallelized.

2.3 Adaptive Strategy for Filter Functions

For the filter functions (`apply_gray_filter`, `apply_blur_filter`, and `apply_sobel_filter`), we implemented a workload-adaptive approach:

- **Image-Level Parallelism:** Applied for GIFs with 8+ images
- **Pixel-Level Parallelism:** Applied for images with at least 1,000 pixels
- **Sequential Processing:** Used for small workloads where parallel overhead would exceed benefits

This strategy dynamically optimizes OpenMP resource utilization based on input characteristics, ensuring efficient execution across diverse GIF files.

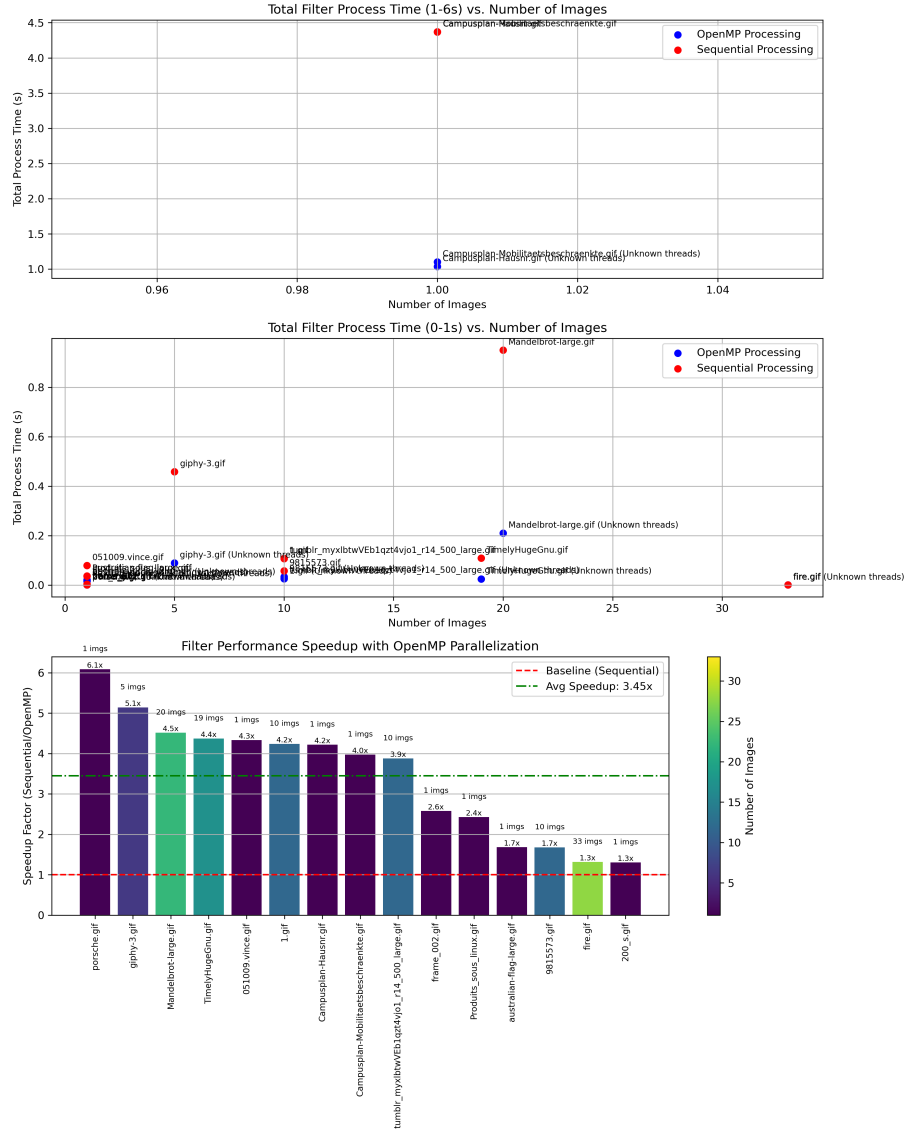


Figure 1: Runtime OpenMP Version

3 CUDA Parallelization Strategy

Our CUDA implementation strategically targets computationally intensive operations that benefit from massive parallelism, while avoiding unnecessary GPU overhead for simpler operations.

3.1 Filter Implementation Selection

We successfully implemented CUDA parallelization for:

- `apply_blur_filter`
- `apply_sobel_filter`

Not all functions were suitable for GPU acceleration. The `apply_gray_filter` has insufficient computational intensity to justify the data transfer overhead. Similarly, `store_pixels` and `load_pixels` involve complex sequential operations and memory patterns that don't map efficiently to GPU architecture.

3.2 Blur Filter Parallelization

For the blur filter, we developed a sophisticated multi-level parallelization approach:

3.2.1 Hierarchical Parallelism

- **Image-Level Parallelism:** Multiple frames processed concurrently using dedicated CUDA streams
- **Pixel-Level Parallelism:** Within each frame, thousands of GPU threads operate on individual pixels using a 2D grid of thread blocks

3.2.2 Region-Based Processing

Our implementation divides each image into three distinct regions:

- **Top Region** (0-10% of height): Processed with blur filter
- **Middle Region** (10-90% of height): Original pixels preserved
- **Bottom Region** (90-100% of height): Processed with blur filter

3.3 Sobel Filter Parallelization

For the Sobel edge detection filter, we developed an efficient GPU implementation:

3.3.1 Parallelization Strategy

- **Image-Level Parallelism:** Multiple frames processed concurrently with dedicated streams
- **Pixel-Level Parallelism:** Thousands of GPU threads operate simultaneously on individual pixels

3.3.2 Algorithm Implementation

- **Stencil-Based Computation:** Each pixel’s gradient calculation uses a 3×3 neighborhood
- **Single-Channel Processing:** The filter operates on the blue channel for efficiency
- **Threshold Application:** Gradient magnitude compared against a fixed threshold

3.4 CUDA Optimization Techniques

We employed several key optimization techniques across our GPU implementations:

- **Asynchronous Execution:** CUDA streams enable concurrent processing of multiple images
- **Kernel Specialization:** Separate kernels for distinct operations improve performance
- **Efficient Memory Management:** Asynchronous memory operations overlap with computation
- **Advanced Resource Cleanup:** CUDA callbacks provide automatic resource deallocation
- **Optimized Thread Blocks:** Fixed 16×16 thread blocks balance occupancy and register usage

3.5 Performance Considerations

Our implementation carefully balances:

- Computational intensity versus data transfer overhead
- Granularity of parallelism (frame-level vs. pixel-level)
- Memory management to minimize bottlenecks
- Asynchronous execution to maximize GPU utilization

This approach effectively leverages GPU hardware capabilities while addressing the specific requirements of animated GIF processing, resulting in significant performance improvements for computationally intensive filters.

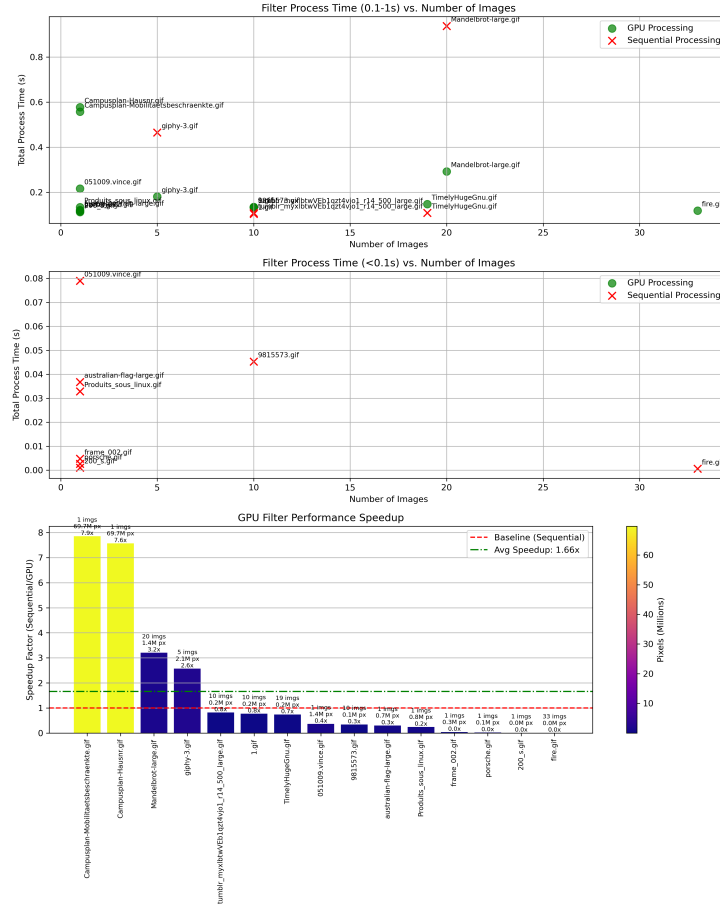


Figure 2: Runtime CUDA Version

4 MPI Parallelization Strategy

Our MPI implementation distributes the workload of processing animated GIF images across multiple parallel processes.

4.1 Initialization and Finalization

- `MPI_Init()` initializes the MPI environment
- Each process retrieves its unique identifier (`rank`) and total processes (`size`) using `MPI_Comm_rank()` and `MPI_Comm_size()`
- `MPI_Finalize()` cleans up the MPI environment when processing is complete

4.2 Work Distribution and Data Communication

- The master process (rank 0) handles input loading and output saving
- Frame counts are broadcasted via `MPI_Bcast()` to synchronize metadata across all processes
- Images are evenly distributed among processes, with careful handling of remainder frames to maintain balanced workloads
- A custom MPI datatype `pixel_type` efficiently communicates pixel data between processes
- The master process sends image dimensions and pixel data to worker processes using `MPI_Send()`, while workers receive this data with `MPI_Recv()`

4.3 Filter Application and Result Gathering

- Processes independently apply grayscale, blur, and Sobel filters to their assigned images
- After processing, worker processes return processed frames to the master via MPI communications
- The master integrates all results into the final GIF output

5 OpenMP-based Sobel Filter Parallelization

The Sobel filter implementation utilizes OpenMP to enhance intra-node parallelism:

5.1 Parallel Region and Loop Parallelization

- An OpenMP parallel region is established with `#pragma omp parallel`
- Nested loops over pixel indices (`j`, `k`) are parallelized with `#pragma omp for collapse(2)` and dynamically scheduled with the `guided` option

5.2 SIMD Vectorization and Memory Management

- Inner loops are vectorized using `#pragma omp simd`, improving memory throughput by processing multiple pixels simultaneously
- Temporary pixel storage arrays are dynamically allocated per image frame and freed after use

5.3 Barrier Synchronization

- A synchronization barrier (`#pragma omp barrier`) ensures completion of computations before results overwrite original data

6 Combined MPI and OpenMP Efficiency

Our implementation exploits hierarchical parallelism effectively:

- **MPI** distributes frames across different nodes or processes
- **OpenMP** ensures optimal utilization of cores within each node

This hierarchical approach enhances scalability and load balancing, making the combined use of MPI and OpenMP particularly suitable for high-performance computing environments.

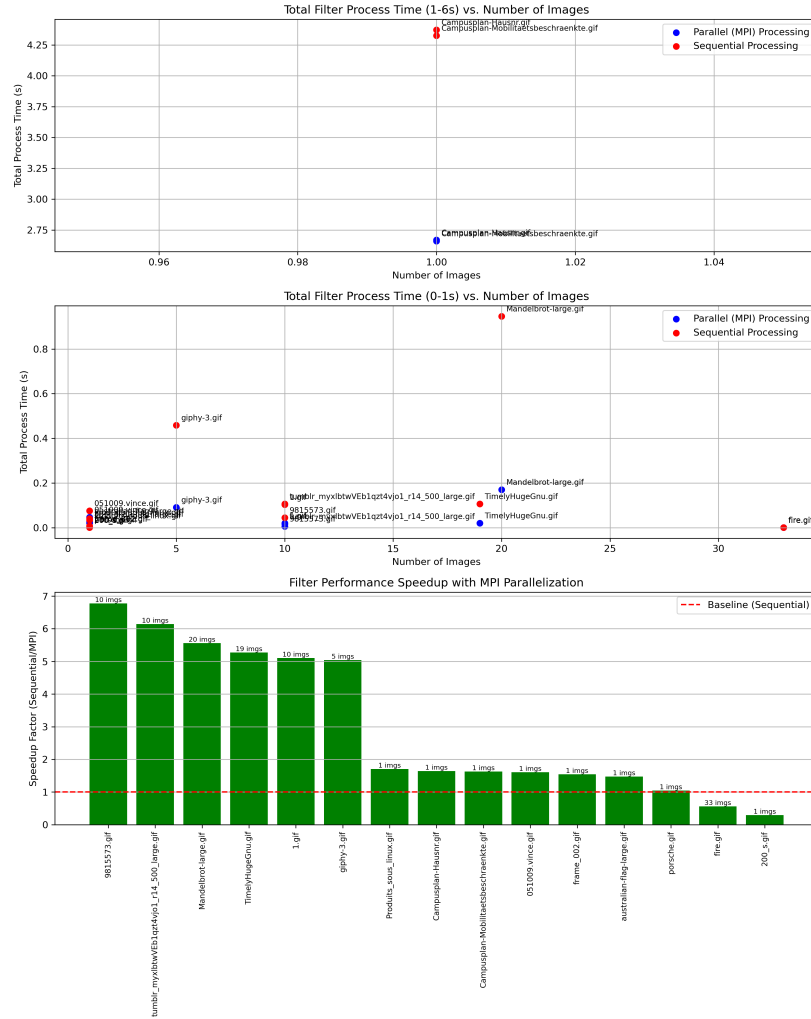


Figure 3: Runtime MPI+OpenMP Version

7 Hybrid CUDA + MPI + OpenMP Implementation

Our final optimized implementation combines the strengths of all three parallelization approaches:

- We retained the MPI implementation for the Sobel filter, where it demonstrated the best performance improvement
- The number of MPI nodes is dynamically determined based on the number of input images
 - If only one image is present, MPI is not used at all
- GPU acceleration via CUDA is utilized for the blur filter when processing images with sufficient pixel count
- OpenMP provides thread-level parallelism where appropriate

This hybrid approach leverages the strengths of each parallelization technique for specific operations, resulting in optimal performance across a wide range of input characteristics.

